Assignment 1

Applied Machine Learning

1. [20 pts] Define each of the following machine learning terms **in your own words**:
   i. the training dataset, testing dataset, and validation dataset
   ii. ground truth, label
   iii. pre-processing, feature, numerical, nominal
   iv. decision surface
   v. model validation, accuracy, cross-validation
   vi. parameters, hyperparameters, overfit
   vii. reclassification

2. [12 pts] Pick the **Iris dataset** of the Scikit-learn datasets for classification which is included in the library (i.e. the dataset can be loaded with `datasets.load_`) and find out the following:
   i. the number of data points
   ii. the number of features and their types
   iii. the number and name of categories (i.e. the `target` field)
   iv. the mean (or mode if nominal) of the first two features

3. [8 pts] Next, locate the **Wine dataset**, load it, and explore it to find out the same characteristics (as in #2.i-iv) once more.

4. [20 pts] Use the following lines of code to display feature pairs from the Iris dataset:

```
import numpy as np
import seaborn as sns; sns.set(style="ticks", color_codes=True)
import sklearn.datasets
import pandas as pd
iris = sklearn.datasets.load_iris()
iris_df = pd.DataFrame(
    data= np.c_[iris.data, [iris.target_names[v] for v in iris.target]],
    columns= iris.feature_names + ['species'])
cols = iris_df.columns.drop('species')
iris_df[cols] = iris_df[cols].apply(pd.to_numeric)
g = sns.pairplot(iris_df, hue='species')
```

   From the plots, which feature(s) shows the most promising separation power for machine learning?

   Now plot the features of the Wine dataset in question 2. When there are too many features, it is possible to switch the dataset or update your code (pandas `Dataframe` line) to look at only a certain number of features at a time.

5. [20 pts] Consider the **Iris dataset**, refer to the plots in the previous question, and discuss/think/outline an unsupervised approach to group the dataset into non-overlapping clusters. Answer the following questions:
   i. Which features would you use?
   ii. Are three clusters obvious from the plots?
   iii. What about four clusters? Either roughly mark them manually (i.e. indicating cutoff thresholds) on a few plots if possible, or specify their ranges (i.e. provide the range of values per cluster per each feature you've chosen to include).
   iv. For this problem, is there any relation between classification and clustering since the labels are already given?

6. [20 pts] Using the `scikit-learn` class descriptions for Naive Bayes and decision trees, classify the Iris dataset from question 4. Your code should be very similar to that in the Module 1 Jupyter notebook. Make sure to divide the dataset into two portions, one for training and the other for testing, as is done in that notebook, using `sklearn.model_selection.train_test_split` (and make sure that shuffle is on).

   Then, answer the following questions:
   i. Why is using shuffle important for this dataset?
   ii. Which classifier has the higher performance?
   iii. Does more training help? Test this by increasing the training dataset size from let's say 1% training (with the remaining 99% testing), and then 5% training, 10% training, etc.
   iv. Will the performance plateau? Show it on a plot.

   Hints
   - Make sure you use the same subset of data each time you train the Naive Bayes classifier as when you train the decision tree classifier.
   - In order to collect statistics (mean and std. dev.), you would need to repeat train-test with different samples e.g., 10 times, for each experimental point. Plotting each experimental data point with mean and std. dev., rather than one performance number, builds a stronger case to tell which classifier is better and how much training is sufficient.